Spec Kit Setup and Guide

Spec Kit Setup

This is the entire documentation on how to setup Spec Kit SDD Toolkit!

What is Specification-Driven Development?

Specification-Driven Development (SDD) is an agentic AI-assisted software development workflow where developers establish clear contracts a.k.a. "specifications" with their AI agents. Unlike you casually prompt your code in which it may cause inconsistencies and misdirection of your project, SDD follows along the planning, implementation, and testing phases which treats like a software engineering companion on hand and it lessens AI hallucinations. The spec artifacts are generated by AI in standard Markdown (`.md`) text file format and saved directly in your local root project folder, making it easy to update and modify.

SDD Core Philosophies:

Detailed: You indicate all of the specifications required in your project. This prevents ambiguities unlike in vibe coding.

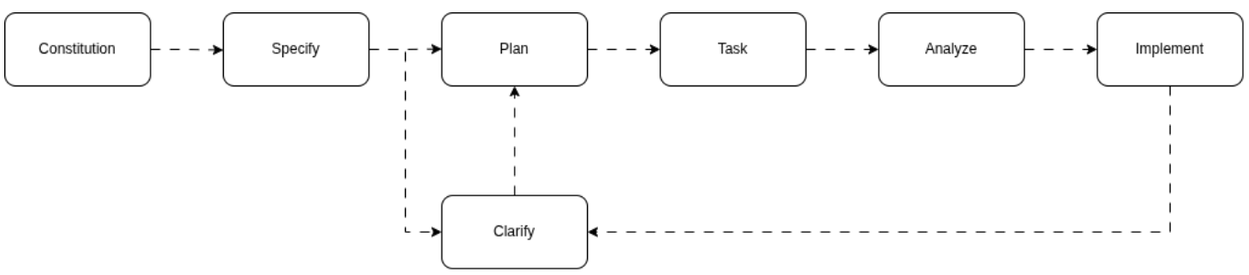
Structured: SDD has structured Markdown (`.md`) file that follows step-by-step across phases and usually stored in `spec\` folder. Nothing will left out.

Behavior-oriented: It focuses on *what* your project should do and *how* should behave from the user's perspective. No need for complicated codes, it prioritizes natural language, just focus on how your project behaves.

AI-Native Partner: This is more than just an code auto completion. Treat it like your coding partner that helps you draft, refine, and execute the specifications.

The SDD Process

These are the key workflow phases in SDD:



Constitution: Specifies the entire foundation of the project.

Example:

- Tech Stack (e.g. HTML, CSS, Java, C++, Tailwind CSS, etc.)
- Tools to be used (e.g. Node.js, REST API, WebGL, etc.)

Specify: Indicates the specification on how the app behaves.

Example:

```
"Create a web application that has a functional dark/light mode on the upper right hand side of the header."
```

Plan: Creates a plan based on the specification indicated by the developer.

Clarify: Checks for clarifications if there are areas in the project that can't understand.

Implement: Transforms the plan into bits of useful tasks.

Validate: The developer verifies the output.

Vibe Coding vs. SDD

Usually, you have a habit of opening your AI agent (e.g., ChatGPT, Gemini, DeepSeek, etc.) and casually chatting with it when you want to explore code or debug errors in your project. But sometimes, the AI creates inconsistencies or falls below your expectations, leaving you unsatisfied with the results. You end up chatting and copy-pasting code all over again, and in the worst-case scenario, it can introduce security risks into your project.

This ad-hoc approach can cause the AI to hallucinate and output out-of-context ideas beyond your project's scope. Prompting blindly like this is often referred to as "vibe coding."

The solution to this problem is **Spec-Driven Development (SDD)**. SDD is a structured, AI-assisted software development workflow that aligns directly with software engineering principles. It transforms ungoverned vibe coding into a governed, structured approach to AI-assisted development.

Vibe Coding	Specification-Driven Development (SDD)
Coined by Andrej Karpathy (2025), this refers to an approach (often called "blind prompting") where you casually chat with AI models to build software without strict context boundaries.	An AI-assisted software development workflow where the AI follows a structured, pre-planned process defined using Markdown (`.md`) text files as the source of truth.

What is Spec Kit?

Is a Specification-Driven Development Toolkit built for agentic AI-assisted software engineering, dedicated for software developers who wants a structured automation in planning, implementation, and testing in their projects.

Official Website: https://speckit.org/
GitHub: https://github.com/github/spec-kit

Prerequisites

`uv` Python Package Manager

An extremely fast, all-in-one Python package and project manager written in Rust and developed by the software company, Astral.

Install `uv` package first before installing Spec-kit.
Link: https://docs.astral.sh/uv/getting-started/installation/
Docs: https://docs.astral.sh/uv/

Run this commands in the terminal to install `uv` on Windows:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

or via PyPI for convenience:

```
pipx install uv
```

or

```
pip install uv
```

Step 1: Install Spec-kit

install Spec-kit by executing these commands in your root project folder with VS Code terminal.

```
uv tool install specify-cli
specify init . --integration <your_ai_agent>
```

Step 2: Continue Installing

A message in the terminal will appear after initializing Spec-kit. Enter `y` to continue.

```
Warning: Current directory is not empty (9 items)
Template files will be merged with existing content and may overwrite existing files. Do you want to continue? [y/N]: y
```

Step 3: Select Coding Agent Integration

Select `opencode` for coding agent integration.

Step 4: Select Script Type

A script type selection will appear, select `ps` if you are using Windows, `sh` for Linux, and `py` for cross OS.

Spec-kit Commands

This is all the slash commands to be used in Spec Kit.

Command	Description	Usage
`/speckit.constitution`	Create project governing principles and development guidelines.	Run first to establish project standards. Use it again if you want to change on the foundational principles.
`/speckit.specify`	Define what you want to build (requirements and user stories)	Focus on **what** and **why**, not tech stack.

Command	Description	Usage
`/speckit.clarify`	Clarify underspecified areas through structured questioning.	Must run before `/speckit.plan` unless explicitly skipped.
`/speckit.plan`	Create technical implementation plans with chosen tech stack.	Specify architecture, frameworks, and technical decisions.
`/speckit.tasks`	Generate actionable tasks lists for implementation.	Breaks down plan into executable steps.
`/speckit.analyze`	Cross-artifact consistency & coverage analysis.	Run after `/speckit.tasks`, before `/speckit.implement`.
`/speckit.implement`	Execute all tasks to build the feature according to plan.	Generates working code from specifications.

Sample Development Workflow

A real life structured approach for optimal results with Spec-Driven Development.

$$Foundation \rightarrow Implementation$$

Phase 1: Foundation

1. **Constitution**: Establish project principles
2. **Specification**: Define requirements clearly
3. **Clarification**: Resole ambiguities

```
/constitution Focus on code quality, testing, UX consistency
/specify Build a photo album organizer with drag-and-drop...
/clarify # AI asks targeted questions
```

Phase 2: Implementation

1. **Planning**: Choose tech stack and architecture.
2. **Tasks**: Break down into actionable items.
3. **Implementation**: Generate working code.

```
/plan Use Vite, vanilla JS, SQLite for local storage
/tasks # Generate task breakdown
/implement # Execute implementation
```

References

Delimarsky, D., & Riem, M. (2026). Spec Kit (Version 1.2.0) [Computer software]. GitHub. https://github.com/github/spec-kit

Kumar, A. (2026). Vibe coding. *Britannica*. https://www.britannica.com/topic/large-language-model